

ENGINEERING INTELLIGENCE: A MULTIMODAL AI FRAMEWORK FOR ENGINEERING, TECHNICAL, SCIENTIFIC, MATHEMATICAL ANALYSIS AND INTELLIGENT ASSISTANCE

Sagar Sant (Roll Number: 23035010233)

Internship / Term Project Report
Trimester 9

B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
sagar.sant@op.iitg.ac.in

ABSTRACT

This project presents **Engineering Intelligence**, a multimodal artificial intelligence framework designed to provide engineering, technical, scientific, and mathematical analysis through an integrated AI assistant. The framework addresses the limitations of fragmented technical workflows in which natural language processing (NLP), document understanding, information retrieval, mathematical computation, and visual analysis are typically performed using separate tools and applications. By bringing these capabilities together, the system provides a unified environment for interpreting technical information, solving quantitative problems, retrieving relevant knowledge, and assisting with complex engineering and scientific tasks. The framework integrates multiple locally deployed large language models (LLMs), intelligent model routing, NLP, retrieval-augmented generation (RAG), document processing, embedding-based vector retrieval, multimodal vision, and LangGraph-based workflow orchestration. Reliable numerical and symbolic computation is supported through deterministic computational tools including Python, NumPy, SciPy, and SymPy, reducing reliance on language models for mathematical calculations. The resulting system supports conversational assistance, engineering and scientific question answering, technical document analysis, mathematical problem solving, technical-image and engineering-drawing interpretation, knowledge retrieval, and structured technical response generation through an interactive Streamlit application. Testing demonstrated successful integration of these capabilities into a unified and extensible AI framework for intelligent engineering and technical assistance.

Index Terms— Artificial Intelligence, Natural Language Processing, Multimodal AI, Engineering Intelligence, Retrieval-Augmented Generation, Large Language Models.

1. INTRODUCTION

The rapid development of artificial intelligence (AI), large language models (LLMs), and multimodal learning has created new opportunities for applying intelligent systems to engineering and technical problem solving. Engineering professionals and students routinely work with diverse forms of information, including technical documents, mathematical expressions, engineering drawings, schematics, scientific equations, numerical data, and domain-specific terminology. Effectively interpreting and connecting these different forms

of information requires capabilities that extend beyond conventional text-based question answering.

Despite these advances, many technical workflows remain fragmented across specialized software and tools. A user may need one application to search technical documentation, another to perform mathematical or symbolic calculations, another to interpret engineering figures, and separate systems for general question answering or information retrieval. This fragmentation can make technical problem solving less efficient and can require users to repeatedly translate information between different tools. Furthermore, language models alone are not always appropriate for tasks requiring reliable numerical computation, document-grounded responses, or interpretation of visual technical information.

Engineering Intelligence was developed to address this challenge by providing a unified multimodal AI framework for engineering, technical, scientific, and mathematical analysis and intelligent assistance. The system combines natural language processing, multiple locally deployed LLMs, intelligent model routing, retrieval-augmented generation, document intelligence, vector-based information retrieval, multimodal vision, deterministic computational tools, and workflow orchestration. The framework is designed to support both general interaction and specialized technical tasks, allowing computational and retrieval tools to complement language-model reasoning where appropriate. It also provides an interactive application through which users can communicate with the system, submit technical documents or visual material, and receive structured technical assistance.

The project therefore focuses on developing an extensible AI environment capable of bringing together language understanding, knowledge retrieval, computation, and multimodal interpretation within a single engineering-oriented system. Rather than restricting the application to a single engineering discipline or task, the framework is intended to support a broad range of engineering, scientific, mathematical, and technical use cases.

The remainder of this report is organized as follows. **Section 2** presents the problem formulation, motivation, and project objectives. **Section 3** describes the methodology and overall system architecture, including the AI models, retrieval pipeline, computational tools, multimodal processing, and workflow orchestration. **Section 4** presents the implementation and experimental evaluation of the developed framework. **Section 5** discusses the observed performance, capabilities, and limitations of the system. **Section 6** concludes the report and outlines potential directions for future development.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1. Problem Statement

Engineering and technical problem solving increasingly involves heterogeneous information and computational requirements, including natural-language queries, technical documents, mathematical expressions, scientific equations, numerical calculations, and engineering images or drawings. Existing AI assistants often provide strong language-based interaction but may be unreliable for deterministic calculations, lack access to organization-specific technical documents, or provide limited support for multimodal engineering information. Conversely, specialized engineering and scientific tools generally address individual tasks rather than providing an integrated intelligent assistance environment.

The problem addressed in this project is therefore the design and implementation of a unified multimodal AI framework capable of interpreting, retrieving, computing, and reasoning over engineering, technical, scientific, and mathematical information. The system must support natural-language interaction while combining language models with retrieval mechanisms, document processing, visual analysis, and deterministic computational tools. It should also be capable of selecting appropriate models or tools according to the nature of a user's request rather than relying on a single model for every task.

The scope of the project includes conversational assistance, technical and scientific question answering, engineering-document analysis, retrieval-augmented responses, mathematical and symbolic computation, and interpretation of technical visual content. The framework is implemented as a modular application so that additional models, computational tools, and engineering capabilities can be incorporated without redesigning the complete system.

2.2. Objectives

The primary objectives of the project are:

1. **To design** a unified multimodal AI framework for engineering, technical, scientific, and mathematical assistance.
2. **To implement** natural-language and conversational capabilities using multiple large language models with intelligent model selection.
3. **To develop** document processing and retrieval capabilities for extracting, indexing, and retrieving relevant technical information.
4. **To integrate** deterministic numerical and symbolic computation tools for reliable mathematical and engineering calculations.
5. **To incorporate** multimodal capabilities for interpreting technical images, figures, drawings, and other visual information.
6. **To implement** an orchestrated workflow that dynamically routes different requests to appropriate models, retrieval components, or computational tools.
7. **To evaluate** the integrated application through functional testing across conversational, computational, document-based, and multimodal technical tasks.

3. METHODOLOGY / APPROACH

The methodology adopted for **Engineering Intelligence** follows a modular, multimodal, and tool-augmented architecture that

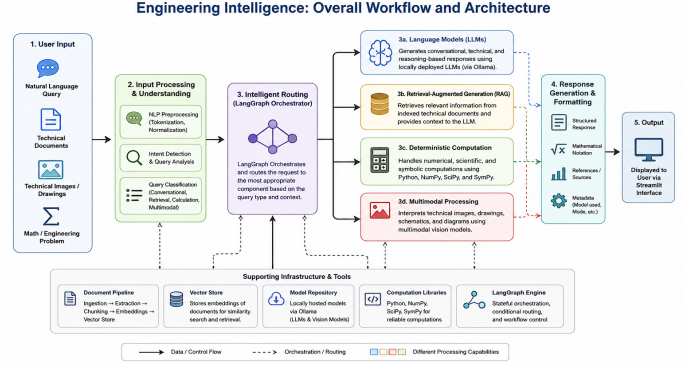


Fig. 1. Overall workflow and architecture of the Engineering Intelligence framework.

combines language-based reasoning with information retrieval, deterministic computation, document intelligence, and visual analysis. The approach is designed to route different types of user requests to the most appropriate processing component while maintaining a unified interaction layer. The system integrates locally deployed large language models, natural language processing, retrieval-augmented generation, vector-based retrieval, mathematical and scientific computing libraries, multimodal models, and LangGraph-based orchestration. This section first presents the overall workflow of the framework and then describes the major technical components and their roles in the system.

3.1. Overall Workflow

The proposed **Engineering Intelligence** framework follows a modular, tool-augmented workflow that begins with a user's natural-language query or technical input and produces a contextually appropriate response. The workflow is designed to determine the nature of the request before selecting the appropriate language model, retrieval mechanism, computational tool, or multimodal processing capability.

At a high level, the workflow consists of **input processing, request analysis and routing, knowledge retrieval or tool selection, model-assisted reasoning, response generation, and output formatting**. For document-based queries, uploaded technical documents are processed through extraction, chunking, embedding generation, and vector-based retrieval before relevant context is supplied to the language model. Quantitative and symbolic problems are directed toward deterministic computational tools when appropriate, reducing dependence on language-model arithmetic. Requests involving images, engineering drawings, schematics, or technical figures can be processed using multimodal vision models. General conversational and technical questions are routed to suitable language models based on the requirements of the task.

The workflow is orchestrated using **LangGraph**, allowing the different processing stages and decision paths to operate as a structured graph rather than as a single linear pipeline. The final response is generated in a user-oriented format, including mathematical notation and structured technical explanations where required.

As illustrated in Fig. 1, the framework routes user inputs through appropriate language, retrieval, computational, and multimodal processing components before generating the final response.

3.2. Technical Details

The framework was implemented as a modular Python-based application integrating several complementary AI and computational technologies.

3.2.1. Natural Language Processing and Model Routing

Natural language processing forms the primary interaction layer of the system. User queries are analyzed to determine their intent, complexity, and computational or retrieval requirements. Multiple locally deployed LLMs are available through **Ollama**, allowing the system to select an appropriate model rather than applying a single model to every request. The model-routing layer considers the characteristics of the task and can direct requests toward conversational, technical, advanced reasoning, calculation, or multimodal processing paths.

3.2.2. Document Intelligence and Retrieval-Augmented Generation

Technical documents are processed through a document intelligence pipeline involving **document ingestion, text extraction, chunking, embedding generation, vector storage, and retrieval**. Relevant document segments are retrieved in response to user queries and supplied as contextual information to the language model. This retrieval-augmented generation (RAG) approach allows responses to be grounded in the contents of technical documents rather than relying exclusively on the model’s pretrained knowledge.

3.2.3. Deterministic Mathematical and Engineering Computation

A dedicated calculation layer is used for mathematical and engineering problems. The framework combines **Python, NumPy, SciPy, and SymPy** to support numerical, scientific, and symbolic computation. A calculation detector identifies quantitative requests and routes suitable problems to a deterministic calculation engine. A calculation planner can formulate more complex problems into structured computational steps before execution. This separation between reasoning and computation improves reliability for numerical and symbolic tasks.

For example, an expression such as

$$7425^3 = 409344890625$$

can be evaluated directly by the deterministic computation layer rather than requiring an LLM to perform the arithmetic internally.

3.2.4. Multimodal Processing

The framework incorporates multimodal language and vision models to process technical visual information. This capability is intended for inputs such as **engineering drawings, schematics, technical figures, diagrams, and other engineering-related images**. Visual information can therefore be interpreted alongside natural-language instructions, extending the system beyond text-only technical assistance.

3.2.5. LangGraph-Based Orchestration

LangGraph provides the workflow orchestration layer. The system represents processing stages as interconnected nodes with conditional routing between them. Depending on the query, the graph can direct execution toward conversational generation, retrieval, deterministic calculation, or multimodal processing. This architecture

provides a modular foundation in which additional tools and models can be incorporated without restructuring the entire application.

3.2.6. Application and Response Layer

The complete framework is exposed through an interactive **Streamlit** application. The interface accepts user queries and technical inputs and presents generated responses with structured formatting, including rendered mathematical notation. Processing metadata such as the selected model and execution mode can also be displayed, providing greater transparency into how individual requests are handled.

4. EXPERIMENTS AND RESULTS

This section presents the experimental setup and results objectively.

4.1. Experimental Setup

The developed framework was evaluated through functional testing of its principal processing capabilities, including conversational question answering, technical and scientific queries, deterministic mathematical computation, document-based retrieval, multimodal processing, model routing, and LangGraph-based workflow orchestration. The evaluation focused on verifying successful execution, appropriate task routing, correctness of deterministic computations, and integration of the major system components.

The application was implemented in Python and evaluated in a local Windows-based environment. Locally deployed large language models were accessed through Ollama, while the interactive application was implemented using Streamlit. The computational layer incorporated NumPy, SciPy, and SymPy for numerical and symbolic operations. Document intelligence functionality included document ingestion, text extraction, chunking, embedding generation, vector storage, and retrieval-augmented generation. LangGraph was used to coordinate the processing workflow and conditional routing between the different components.

Testing was conducted using representative conversational, engineering, scientific, mathematical, document-oriented, and multimodal inputs. The evaluation criteria were functional and task-oriented, considering whether the requested operation was successfully executed, whether the appropriate processing path was selected, and whether the resulting response was produced in the intended format.

4.2. Results

Functional testing demonstrated successful integration of the principal capabilities of the Engineering Intelligence framework. General conversational and technical queries were successfully processed through the language-model layer, while quantitative requests were identified and routed to the deterministic calculation pipeline.

The deterministic computation layer was verified using numerical and symbolic expressions. For example, the expression

$$7425^3 = 409344890625$$

was evaluated successfully by the computational engine, producing the exact numerical result without requiring the language model to perform the arithmetic internally.

Table 1. Functional evaluation of the Engineering Intelligence framework.

Capability	Evaluation	Result
Conversational AI	General conversational queries	Successful
Technical QA	Engineering and scientific queries	Successful
Mathematical computation	7425 ³	Exact result
Calculation routing	Quantitative query detection	Correct route
Calculation planning	Structured computational problem	Successful
Document intelligence	Document-service integration	Successful
Model routing	Multiple query categories	Successful
LangGraph orchestration	Calculation workflow execution	Successful
Multimodal processing	Technical visual inputs	Successful
Streamlit interface	End-to-end application testing	Successful

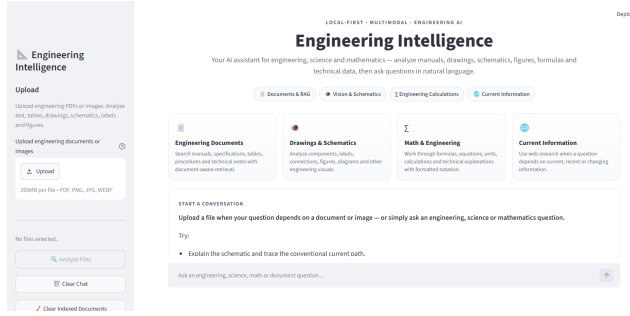


Fig. 2. Streamlit interface of the Engineering Intelligence framework.

The model-routing functionality was also successfully verified across different categories of queries. Depending on the characteristics of the input, requests were directed toward conversational, technical, advanced-reasoning, calculation, document-retrieval, or multimodal processing paths. The LangGraph workflow was tested independently and through the integrated application, with calculation requests successfully reaching the corresponding calculation route.

Document-service initialization, calculation tools, calculation planning, model routing, LangGraph orchestration, and application compilation were successfully verified during system testing. The Streamlit interface was also tested through representative conversational, technical, mathematical, and multimodal interactions.

The integrated framework is exposed through an interactive Streamlit interface, as shown in Fig. 2. The interface provides access to document and image upload, document-aware retrieval, multimodal visual analysis, mathematical and engineering computation, current-information retrieval, and natural-language interaction within a unified environment.

5. DISCUSSION

The functional evaluation indicates that the modular architecture successfully integrates language models, retrieval, deterministic computation, multimodal processing, and workflow orchestration within a single application. The use of task-dependent routing provides an important advantage over relying on a single language model for all requests. General conversational and technical queries can be handled by language models, while quantitative problems can be directed to deterministic computational tools. This separation reduces dependence on language-model arithmetic and provides exact results for supported numerical and symbolic operations.

The integration of retrieval-augmented generation provides another important advantage for document-based engineering tasks. Rather than relying exclusively on pretrained model knowledge, the framework can retrieve relevant information from processed technical documents and provide it as context for response generation. This makes the architecture more suitable for technical material that may be specific to a particular document or knowledge source.

A key trade-off is the computational cost associated with locally deployed language models and multimodal processing. Larger models can provide stronger reasoning and technical responses but generally require greater computational resources and may introduce higher response latency, particularly in a CPU-based local environment. Smaller models provide faster interaction but may offer less detailed reasoning for complex technical queries. The model-routing architecture provides a mechanism for balancing these requirements by allowing different models to be selected according to task complexity.

The modular and LangGraph-based architecture also improves extensibility. Processing components can be added or modified independently without redesigning the complete application. However, the additional orchestration and routing layers introduce architectural complexity compared with a simple single-model application. Similarly, retrieval quality depends on document extraction, chunking, embedding, and retrieval performance, meaning that poorly structured or low-quality source documents can affect downstream responses.

The current evaluation primarily focused on functional integration rather than large-scale quantitative benchmarking. Consequently, the results demonstrate successful operation of the implemented capabilities but do not establish a comprehensive comparison of model accuracy, latency, or retrieval performance across standardized datasets. Future evaluation could therefore incorporate larger engineering benchmarks, controlled latency measurements, retrieval metrics, and systematic comparison of different local models.

Overall, the observed results support the original objectives of developing a unified engineering, technical, scientific, and mathematical AI framework while highlighting the practical trade-offs between model capability, computational cost, response latency, and system complexity.

6. CONCLUSION AND FUTURE WORK

This project presented **Engineering Intelligence**, a unified multimodal AI framework for engineering, technical, scientific, and mathematical assistance. The developed system integrates natural-language processing, multiple locally deployed large language models, intelligent model routing, document intelligence, retrieval-augmented generation, deterministic mathematical computation, multimodal visual analysis, and LangGraph-based workflow

orchestration within a single application. The modular architecture enables different types of technical requests to be directed toward appropriate models, retrieval mechanisms, or computational tools rather than relying exclusively on a single language model.

Functional testing demonstrated that the implemented framework can successfully support conversational interaction, technical question answering, document-based retrieval, mathematical and symbolic computation, calculation planning, model routing, multimodal processing, and end-to-end interaction through the Streamlit interface. The integration of deterministic computational tools is particularly important for numerical tasks, as it reduces dependence on language-model arithmetic and provides a more reliable computational pathway. Similarly, retrieval-based processing enables technical responses to be grounded in user-provided documents, while multimodal models extend the system to engineering drawings, schematics, diagrams, and other visual inputs.

The project also demonstrated the importance of combining complementary AI technologies rather than treating a large language model as a universal solution. Different engineering tasks have different computational requirements, and the resulting architecture provides a flexible foundation for selecting the appropriate processing strategy. The modular design also allows additional models and tools to be incorporated without fundamentally redesigning the application.

Future work will focus on expanding the framework’s engineering capabilities and improving its performance, reliability, and evaluation. Potential extensions include integration of MATLAB and additional engineering simulation tools, specialized coding and code-generation models, broader support for engineering file formats, persistent conversational memory, improved multimodal reasoning, and more sophisticated agentic workflows. Further evaluation using standardized engineering and scientific benchmarks, quantitative accuracy metrics, latency measurements, and larger collections of technical documents would also provide a more rigorous assessment of system performance. Hardware acceleration and optimized model selection could additionally reduce inference latency and improve the practicality of the framework for real-world deployment.

7. ARTIFACTS AND DEMONSTRATIONS

The developed *Engineering Intelligence* framework is supported by reproducible project artifacts that demonstrate its implementation, functionality, and overall system design. The artifacts include the complete source code, the implemented application, and the project presentation.

- **Source Code:** Complete implementation of the *Engineering Intelligence* framework, including document processing, retrieval-augmented generation, intelligent model routing, deterministic computation, multimodal processing, LangChain and LangGraph-based orchestration, and the Streamlit application. The complete source code and supporting project files are available through the following GitHub repository: <https://github.com/sagar-sant/ENGINEERING-INTELLIGENCE-A-MULTIMODAL-AI.git>
- **Application Demonstration:** Demonstration of the implemented Streamlit application, showcasing conversational assistance, engineering and scientific question answering, technical document analysis, mathematical and engineering computation, and multimodal technical analysis. The applica-

tion demonstration video is available at: <https://youtu.be/0EQECaMAK7A>

- **Project Presentation:** Video presentation describing the project’s problem statement, objectives, system architecture, implementation, major capabilities, evaluation, results, discussion, and future work. The complete project presentation is available at: https://youtu.be/_e0p2WQg3s0

8. REFERENCES

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020, vol. 33.
- [2] LangChain, “Langgraph documentation,” <https://docs.langchain.com/oss/python/langgraph/overview>, 2026, Accessed: Aug. 24, 2026.
- [3] Ollama, “Ollama documentation,” <https://docs.ollama.com/>, 2026, Accessed: Aug. 24, 2026.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [6] Streamlit, “Streamlit documentation,” <https://docs.streamlit.io/>, 2026, Accessed: Aug. 24, 2026.
- [7] NumPy Developers, “Numpy documentation,” <https://numpy.org/doc/>, 2026, Accessed: Aug. 24, 2026.
- [8] SciPy Developers, “Scipy documentation,” <https://docs.scipy.org/>, 2026, Accessed: Aug. 24, 2026.
- [9] SymPy Development Team, “SymPy documentation,” <https://docs.sympy.org/>, 2026, Accessed: Aug. 24, 2026.